LINUX / SYSTEMS

Bash Scripting Visual Guide

Pipes, redirection, expansions, quoting, strict mode, and functions.

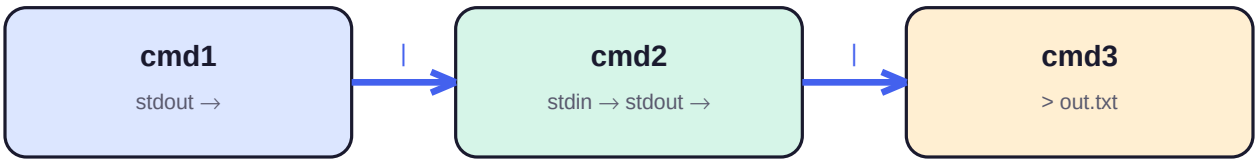
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Pipes & Redirection: Plumbing Data

stdout becomes someone else's stdin; files catch the streams.



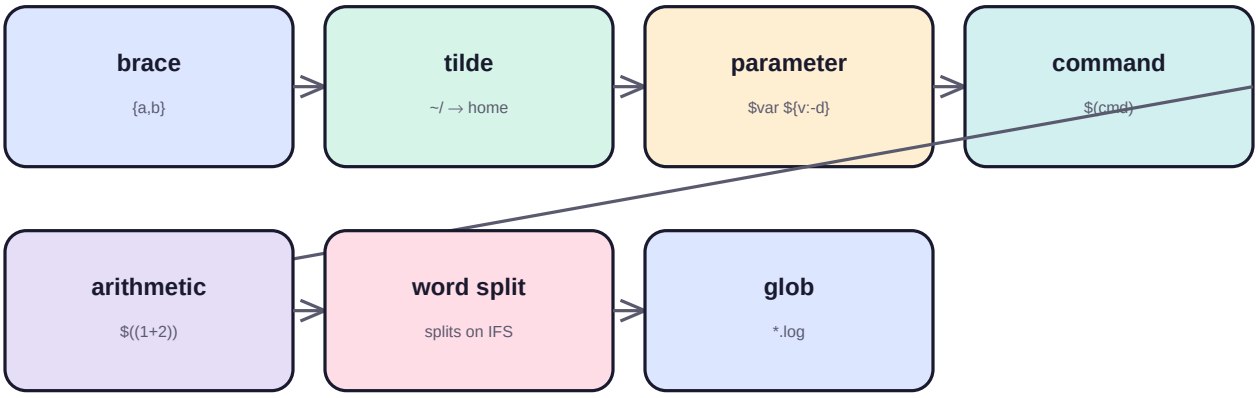
FD	Name	Redirect
0	stdin	< in.txt
1	stdout	> out.txt (>> appends)
2	stderr	2> err.log
1+2	both	> all.log 2>&1 (order matters!)

2>&1 must come AFTER the file redirect: '> f 2>&1' merges into f; '2>&1 > f' sends stderr to the OLD stdout (the terminal). Order is everything.

STEP 2/6

Expansions: Bash Rewrites Your Line First

Seven expansions run before the command executes — in order.




```
files=(/var/log/*.log)     # glob expands NOW, safely into array
echo "count: ${#files[@]}"
name=${1:-world}           # default if unset — see step 4
```

STEP 3/6

Quoting: Three Levels of Protection

Unquoted → split + glob. Double → expand but don't split. Single → literal.

Quoting	$var	$(cmd)	*.log	Use when
none	expands	runs	globs	you WANT splitting (rare)
"double"	expands	runs	literal	ALMOST ALWAYS — default choice
'single'	literal	literal	literal	regexes, awk, literal dollars

```
f="my file.txt"; rm "$f"     # quoted: one arg. Unquoted: TWO args!
echo '$HOME'  # prints $HOME — single quotes = literal
echo "$HOME"  # prints /home/enmanuel
```

Rule: quote every variable expansion unless you can state why it must split. ShellCheck (shellcheck.net) enforces this automatically — run it on every script.

STEP 4/6

Strict Mode: set -euo pipefail

Four switches that turn silent corruption into loud failure.

-e	-u	-o pipefail	-x*
exit on any failed command	exit on use of UNSET variable	pipeline fails if ANY stage fails	trace (debug only)

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
name=${1:?"usage: $0 <name>"}   # -u friendly required arg
```

Gotchas: -e ignores failures in `if` conditions and `||` fallbacks (by design). grep finding nothing exits 1 — guard it: grep -q x f || true.

STEP 5/6

Functions: Named, Testable Blocks

local keeps variables from leaking; return carries status, echo carries data.

```
backup() {
  local src=$1 dest=$2          # local: no globals polluted
  [[ -d $src ]] || { echo "no src" >&2; return 1; }
  rsync -a "$src/" "$dest/"
}
backup /data /backup || die "backup failed"
```

- Functions return an EXIT STATUS (0-255), not strings — print data to stdout, signal with return.
- local in every function; quote every parameter. Globals are a bug waiting for a second caller.
- die() { echo "ERROR: $*" >&2; exit 1; } — one error handler beats twenty inline exits.

STEP 6/6

Debugging Toolkit

See what bash sees.

```
bash -x script.sh            # trace every expansion (noisy, honest)
set -x / set +x              # trace one section only
trap 'echo "failed at line $LINENO"' ERR
shellcheck script.sh         # static analysis — catches 90% of bugs
bash -n script.sh            # syntax check without running
```

- Reproduce with -x first: most 'impossible' bugs are an unexpected expansion.
- trap ERR + $LINENO pinpoints the failing line in long scripts.
- Put shellcheck in CI — it flags unquoted vars, bad test syntax, and pipefail gaps.

STEP ✓/✓

Key Takeaways

Bash Scripting

- Plumbing: pipes chain stdout→stdin; 2>&1 order matters — redirect the file first.
- Seven expansions rewrite each line before execution; quote to control them.
- Default to double quotes on every expansion; single quotes for literals.
- Start every script with set -euo pipefail — fail loud, fail early.
- Functions: local vars, echo for data, return for status; debug with -x and shellcheck.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev